Diagnóstico Automatizado de Falhas em Microsserviços via Agentes AIOps

Resumo

A adoção de arquiteturas de microsserviços em ambientes *Cloud-Native* trouxe complexidade operacional significativa: uma única transação de usuário pode percorrer dezenas de serviços independentes, gerando volumes de telemetria que excedem a capacidade de análise humana. Agentes AIOps baseados em *Large Language Models* [LLMs] aparecem como caminho para automatizar a *Root Cause Analysis* [RCA] de incidentes em sistemas distribuídos, mas sua operacionalização enfrenta dois entraves: a perda de precisão por falsos positivos (*Alert Fatigue*) e o impacto econômico do consumo de tokens em APIs comerciais. Este trabalho apresentou uma análise comparativa entre duas arquiteturas de ativação de agentes AIOps, *Event-Driven* (reativa, acionada por alertas do Prometheus) e *Continuous Polling* (proativa, com varredura cíclica de logs a cada trinta segundos), ambas empregando o modelo Gemini 2.5 Flash. O experimento foi conduzido sobre a Google Online Boutique, com cinco cenários de Engenharia do Caos executados cinco vezes por arquitetura, totalizando cinquenta execuções. A avaliação empregou cinco critérios de qualidade SRE (escala 0–15 pontos) e cinco métricas FinOps. O agente *Event-Driven*, com alertas abrangentes e leitura *cross-service* de *logs*, apresentou superioridade nesse trabalho em relação ao *Continuous*. A conclusão central é que a qualidade da observabilidade constitui o fator determinante da eficácia de agentes AIOps baseados em LLM.

Palavras-chave: AIOps; Microsserviços; Large Language Models; Engenharia de Confiabilidade de Sites; FinOps.

Automated Fault Diagnosis in Microservices via AIOps Agents

Abstract

The adoption of microservices architectures in Cloud-Native environments has introduced significant operational complexity, where a single user transaction may traverse dozens of independent services, generating telemetry volumes that exceed human analytical capacity. AIOps agents powered by Large Language Models [LLMs] emerge as an alternative for automating Root Cause Analysis [RCA] in distributed systems. However, their operationalization faces two central challenges: precision degradation due to false positives (Alert Fatigue) and the economic impact of token consumption in commercial APIs. This work presented a comparative analysis between two AIOps agent activation architectures, Event-Driven (reactive, triggered by Prometheus alerts) and Continuous Polling (proactive, with cyclic log scanning every thirty seconds), both employing the Gemini 2.5 Flash model. The experiment was conducted on Google Online Boutique, with five Chaos Engineering scenarios executed five times per architecture, totaling fifty runs. Evaluation employed five SRE quality criteria (0–15 points scale) and five FinOps metrics. The Event-Driven agent, configured with comprehensive alerts and cross-service log collection, achieved best results compared to the Continuous agent. The central conclusion is that the quality of the underlying observability constitutes the determining factor for LLM-based AIOps agent effectiveness.

Keywords: AIOps; Microservices; Large Language Models; Site Reliability Engineering. FinOps.

Introdução

A engenharia de software consolidou, ao longo da última década, a transição de aplicações monolíticas para ecossistemas de microsserviços, nos quais cada domínio funcional é encapsulado em um serviço autônomo com ciclo de vida independente (NEWMAN, 2021). Essa quebra, aliada à adoção de infraestruturas *Cloud-Native* sustentadas por contêineres, trouxe benefícios em escalabilidade e isolamento de falhas, mas aumentou a complexidade operacional. Em um ambiente de microsserviços, uma transação aparentemente trivial pode percorrer dezenas de serviços, cada qual gerando *logs*, métricas e sinais de rastreamento. Segundo Majors, Fong-Jones e Miranda (2022), a observabilidade,

entendida como a capacidade de compreender o estado interno de um sistema a partir de suas saídas externas, torna-se progressivamente mais difícil de satisfazer nesse contexto. Quando o volume de telemetria excede a análise humana, a disciplina de AIOps (*Artificial Intelligence for IT Operations*) ganha relevância. Com a ascensão dos LLMs generativos, agentes AIOps passaram a interpretar *logs* contextualmente, correlacionar eventos entre serviços e gerar relatórios de causa raiz de forma autônoma, abrindo um campo de investigação novo: agentes que atuem como analistas de confiabilidade automatizados.

A aplicação prática de LLMs como agentes operacionais enfrenta dois entraves. O mais crítico é a *Alert Fatigue*: quando o monitoramento dispara notificações falsas, a equipe de plantão se dessensibiliza, comprometendo a confiança no sistema (BEYER et al., 2018). LLMs agravam esse risco ao interpretar literalmente a severidade nominal dos logs, de modo que um registro classificado como *error*, mesmo inofensivo, pode ser diagnosticado como anomalia. Não menos importante há a esfera financeira: LLMs comerciais são precificados por *token*, e enviar milhares de linhas de *log* a cada ciclo gera custo proporcional ao volume. Em uma estratégia de varredura contínua, chamadas durante períodos saudáveis constituem despesas sem retorno diagnóstico.

Na prática, a integração de IA generativa ao monitoramento divide-se em algumas possibilidades. O modelo Proativo (*Continuous Polling*) realiza varreduras cíclicas em intervalos fixos, buscando anomalias antes que alertas tradicionais disparem, incorrendo em custo fixo por ciclo e exposição constante a ruído. O modelo Reativo (*Event-Driven*) mantém o agente adormecido até que *thresholds* de infraestrutura sejam violados, oferecendo custo nulo em operação normal, mas dependendo integralmente dos alertas configurados.

A complexidade dos ambientes modernos supera a capacidade de equipes de *Site Reliability Engineering* [SRE] de conduzir RCA manual com a velocidade exigida (GARTNER, 2023), e validar qual estratégia de ativação preserva a credibilidade do agente é contribuição direta à maturidade de SRE. No aspecto financeiro, mensurar a diferença de custo entre invocar o LLM apenas sob evidência de anomalia versus invocá-lo a cada ciclo oferece dados concretos para dimensionamento. Incluso, o trabalho contribuiu ao propor um experimento controlado que avaliou qualidade diagnóstica e custo de forma conjunta, demonstrando que a configuração de observabilidade, e não a arquitetura de ativação, constitui o fator determinante da eficácia.

O objetivo geral foi analisar comparativamente a eficácia diagnóstica e o custo operacional de duas arquiteturas de agentes AIOps, *Event-Driven* e *Continuous Polling*, fundamentadas no modelo Gemini 2.5 Flash, em um ambiente de microsserviços submetido a cenários controlados de falha. Os objetivos específicos foram: (a) implementar dois agentes com escopo equalizado de monitoramento *cross-service*; (b) projetar e executar cinco

cenários de Engenharia do Caos com complexidade progressiva; (c) elaborar e aplicar critérios de qualidade fundamentados em princípios de SRE; e (d) mensurar as métricas FinOps associadas ao consumo de *tokens* em ambas as arquiteturas.

Metodologia

Fundamentação Teórica

Arquiteturas de Microsserviços e Aplicações Cloud-Native

Em microsserviços, a aplicação é dividida em serviços pequenos, autônomos e fracamente acoplados, cada um responsável por uma capacidade de negócio (NEWMAN, 2021). Essa abordagem contrasta com a arquitetura monolítica, na qual todos os módulos são implantados como uma unidade e qualquer nova funcionalidade exige recompilar e reimplantar toda a aplicação. Na arquitetura distribuída, cada serviço comunica-se com os demais via protocolos de rede, predominantemente REST (HTTP/JSON) e gRPC (HTTP/2 com Protocol Buffers) (BURNS, 2018). Os benefícios são bem documentados: isolamento de falhas, escalabilidade granular e autonomia tecnológica. A Google Online Boutique, utilizada neste trabalho, exemplifica essa diversidade, composta por serviços escritos em Go, Python, Java, C# e Node.js (GOOGLE CLOUD, 2023).

Essa flexibilidade, no entanto, cobra um preço operacional. Soldani et al. (2018) identificaram que os desafios mais reportados em microsserviços são a complexidade de *debugging* em chamadas distribuídas, a dificuldade de manter consistência de dados e a sobrecarga de gerenciar dezenas de serviços em produção. A complexidade de *debugging* constitui o eixo central do problema investigado neste trabalho: quando uma requisição de checkout percorre oito serviços em sequência, a identificação do ponto de falha exige instrumentação em cada etapa. A evolução natural dos microsserviços conduziu ao conceito de aplicações *Cloud-Native*, formalizado pela CNCF, que utilizam contêineres, orquestração e infraestrutura imutável. Nesse ecossistema, a operação torna-se um problema de engenharia de software, premissa que fundamenta a disciplina de SRE.

Observabilidade em Sistemas Distribuídos

A observabilidade designa a capacidade de compreender o estado interno de um sistema a partir de suas saídas externas (MAJORS; FONG-JONES; MIRANDA, 2022). Diferencia-se do monitoramento tradicional, que responde a perguntas pré-definidas ("a CPU está acima de 80%?"), ao aspirar oferecer dados suficientes para perguntas que emergem apenas no momento do incidente.

Para observabilidade, métricas são dados numéricos agregados (CPU, memória, latência, taxa de erros) que constituem a base de sistemas de alerta como o Prometheus. *Logs* são registros textuais de eventos individuais, ricos em contexto (*timestamps*, severidade, identificadores de sessão), mas ambíguos, pois um registro com ***severity: "error"*** pode indicar falha real ou ser um registro esperado do ambiente. *Traces* distribuídos representam o percurso completo de uma requisição entre serviços, sendo o pilar mais poderoso para diagnóstico, mas também o mais complexo de implementar.

A interação entre os pilares revela um desafio recorrente: suas lacunas. Há uma lacuna quando um componente não emite sinais suficientes para que seu comportamento seja entendido externamente.

Engenharia de Confiabilidade de Sites (SRE)

A SRE é uma disciplina formalizada pela Google que trata a operação de sistemas como problema de engenharia de software (BEYER et al., 2016). Apoia-se em conceitos como *SLI* (métrica quantitativa de saúde do serviço), *SLO* (alvo definido para o SLI) e *Error Budget* (margem de indisponibilidade tolerada que viabiliza inovação). No centro das práticas operacionais encontra-se a *Root Cause Analysis* (RCA): o processo investigativo que identifica não apenas sintomas, mas a origem de uma degradação. Em microsserviços, uma RCA eficaz exige correlacionar sinais de múltiplos serviços e reconstituir o caminho da requisição que falhou.

Um desafio intimamente relacionado é a *Alert Fatigue*. Acontece quando o volume de notificações ultrapassa a capacidade da equipe de absorvê-las, levando à dessensibilização. Beyer et al. (2018) documentaram que a deterioração da confiança no sistema de alertas é um dos fatores mais destrutivos para a cultura de resposta a incidentes. A *Alert Fatigue* manifesta-se das seguintes formas: alertas para condições tecnicamente anômalas, mas sem impacto ao usuário, e alertas repetitivos sem contextualização ou supressão de padrões já conhecidos pelo time.

AIOps e LLMs Generativos

O termo AIOps foi cunhado pela Gartner (2023) para designar a aplicação de inteligência artificial à automação de operações de TI. A evolução da área ocorreu em três gerações: regras estáticas com *thresholds* fixos, técnicas de aprendizado de máquina clássico (LI et al., 2022; NOTARO; CARDOSO; GERNDT, 2020), e, mais recentemente, LLMs generativos. Esta terceira geração representou uma ruptura: LLMs são redes neurais com bilhões de parâmetros, treinados sobre código, documentação e *logs*, que oferecem capacidades inéditas, como interpretação semântica desses *logs* (compreender que *"Credit*

card info is invalid" é falha de validação, não de infraestrutura), correlação textual entre serviços (identificar que um *session* ID no serviço A corresponde ao *user_id* no serviço B), geração de relatórios de RCA em linguagem natural, e classificação contextual de severidade.

As limitações inerentes dos LLMs incluem a natureza probabilística (respostas variam entre invocações), ausência de memória entre chamadas, risco de alucinação e interpretação literal de severidade de *logs*. Essas limitações não invalidam sua aplicação, mas delimitam as condições de eficácia, condicionada pela qualidade dos dados fornecidos e pela arquitetura que determina quando o modelo é invocado.

Modelos de Ativação de Agentes

A literatura descreve algumas abordagens para integrar agentes AIOps ao ecossistema de observabilidade. No modelo *Event-Driven* (reativo), o agente só ativa quando um alerta indica violação de *threshold*. A vantagem central é a precisão de ativação, com custo zero em operação normal e alta especificidade. A limitação é a dependência integral dos alertas configurados: falhas que não violam *thresholds* permanecem invisíveis. No modelo *Continuous Polling* (proativo), o agente analisa *logs* em intervalos fixos independentemente de alertas. O ponto forte é a amplitude de cobertura para anomalias sutis. As limitações são o custo fixo por ciclo, a tendência a confundir ruído com anomalias (sem mecanismo de contextualização, o LLM interpreta registros ambientais como problemas), e a cegueira de infraestrutura, já que o agente lê apenas *logs* textuais, sem acesso a métricas de CPU ou memória.

Os dois modelos não são necessariamente excludentes. Abordagens híbridas, em que o *Event-Driven* age como primeiro atuante e o *Continuous* opera como rede de segurança com intervalos mais espaçados, podem combinar a especificidade do primeiro com a amplitude do segundo.

Métodos

Engenharia do Caos

A Engenharia do Caos consiste na introdução deliberada de falhas controladas em um sistema para verificar sua resiliência (ROSENTHAL; JONES, 2020). As falhas neste trabalho dividem-se em camada de infraestrutura (L4), como exaustão de CPU e OOM, que se manifestam em métricas quantitativas, e camada de aplicação (L7), como dados inválidos e incompatibilidades, que se manifestam em logs sem impacto imediato em métricas. Essa categorização é central ao trabalho, pois cada camada desafia os agentes de forma distinta:

falhas L4 ativam naturalmente o *Event-Driven*, enquanto falhas L7 são detectáveis apenas pela análise textual de *logs*.

Para orquestração, utilizou-se o K6 (Grafana Labs), framework de testes de carga que gera carga controlada e reprodutível, produz métricas de performance automaticamente, e oferece flexibilidade de *script* para cenários complexos. O K6 atuou como gerador de carga que provoca indiretamente as condições de falha desejadas. Essa abordagem é menos precisa que a injeção direta de falhas, mas também oferece realismo.

FinOps

FinOps é uma disciplina que promove colaboração entre engenharia e finanças para otimizar gastos com nuvem (FINOPS FOUNDATION, 2023). O princípio central é que cada decisão técnica é, simultaneamente, uma decisão financeira. No contexto deste trabalho, os princípios de FinOps foram adaptados ao consumo de APIs de IA generativa. Cinco métricas foram definidas: **FIN-1**, Custo Total por Execução (custo em dólares de todas as chamadas ao LLM); **FIN-2**, Custo por Diagnóstico Útil (custo dividido pelo número de diagnósticos que identificaram anomalias reais); **FIN-3**, Latência Média de Diagnóstico (tempo entre processamento e publicação); **FIN-4**, Taxa de Relevância de Alertas (proporção de alertas que corresponderam a anomalias reais); e **FIN-5**, Overhead de Tokens Ociosos (proporção de tokens consumidos sem valor diagnóstico, exclusiva do *Continuous*).

Caracterização da Pesquisa

Tratou-se de uma pesquisa aplicada, com abordagem quali-quantitativa e caráter exploratório-descritivo. O procedimento foi experimental, estruturado em torno de duas variáveis independentes: (i) a arquitetura de ativação do agente (*Event-Driven* ou *Continuous Polling*) e (ii) o cenário simulado (cinco níveis: Baseline, Error Spike, Chaos Spike, CPU Spike e OOM Test). As variáveis dependentes foram a qualidade diagnóstica (mensurada pelos critérios de qualidade, de 0 a 15 pontos) e o custo operacional (mensurado pelas métricas FinOps). Cada combinação (duas arquiteturas × cinco cenários) foi executada cinco vezes, totalizando cinquenta execuções experimentais.

Ambiente Experimental

- Aplicação-Alvo: Google Online Boutique.

A aplicação utilizada foi a Google Online Boutique (v0.8.0), e-commerce de demonstração mantido pela Google Cloud Platform como referência para arquiteturas de microsserviços (GOOGLE CLOUD, 2023), composta por onze microserviços em cinco

linguagens. O fluxo experimental relevante foi o de *checkout*: acesso à homepage (*GET /*), adição ao carrinho (*POST /cart*) e finalização (*POST /cart/checkout*). Esta última requisição aciona chamadas gRPC internas: *frontend → checkoutservice → paymentservice, shippingservice, cartservice e emailservice*. Uma falha em qualquer ponto se propaga como erro gRPC até o frontend.

A escolha justificou-se pela ampla utilização em estudos acadêmicos, pela diversidade de protocolos (HTTP + gRPC) que introduz complexidade realista, e pela cadeia de dependências no *checkout* que oferece superfície rica para Engenharia do Caos.

- Infraestrutura de Orquestração:

O ambiente foi orquestrado por Docker Compose em uma instância Google Compute Engine (GCE), totalizando dezesseis contêineres: os onze serviços da aplicação, três componentes de monitoramento (Telegraf, Prometheus, Alertmanager) e os dois agentes AIOps. Para simular condições de exaustão, foram configurados limites explícitos nos contêineres do fluxo de checkout:

Tabela 1. Limites dos contêineres do fluxo de checkout

Contêiner	CPU Limit (vCPU)	Memory Limit (MiB)
checkoutservice	0.25	100
frontend	0.50	200

Fonte: Dados originais da pesquisa

A escolha de Docker Compose em detrimento de Kubernetes foi proposital. Em Kubernetes, mecanismos como *Horizontal Pod Autoscaler* e *pod scheduling* introduziriam complexidade que dificultariam o isolamento das variáveis independentes. O Docker Compose ofereceu um ambiente mais determinístico, com limites fixos e sem escalonamento automático.

- Stack de Observabilidade

O Telegraf monitorou todos os contêineres via socket Docker, coletando métricas de CPU, memória, rede e I/O, expostas para *scraping* pelo Prometheus. Quatro regras de alerta foram configuradas no Prometheus:

Tabela 2. Visão de regras de alerta do Prometheus

Regra	Contêiner	Métrica	Threshold	Severidade
HighCpuUsage_CheckoutService	checkoutservice	CPU	> 5%	warning
HighMemory_OOM_Risk	checkoutservice	Memória	> 90%	critical
HighCpuUsage_Frontend	frontend	CPU	> 5%	warning
HighMemory_OOM_Risk_Frontend	frontend	Memória	> 90%	critical

Fonte: Dados originais da pesquisa

As regras de CPU incluíram uma cláusula *unless* que suprime o alerta quando a memória cruza 90%, priorizando o alerta de memória. Os *thresholds* de CPU (5%) foram deliberadamente baixos para garantir ativação em todos os cenários de carga. O Alertmanager roteou as notificações via webhook HTTP ao agente *Event-Driven*.

Implementação dos Agentes AIOps

Ambos os agentes foram desenvolvidos em Python e utilizaram o Gemini 2.5 Flash via biblioteca *google-generativeai*. Os diagnósticos foram publicados em canais dedicados do Discord via *webhooks*. O código-fonte completo está disponível no **repositório desse projeto**.

O agente *Event-Driven* operou como aplicação *Flask* com *endpoint* HTTP POST. Ao receber um *webhook* do Alertmanager, o agente validava o alerta, aplicava deduplicação por *cooldown* de cinco minutos, coletava as últimas duzentas linhas de log de ambos os contêineres monitorados (*frontend* e *checkoutservice*) via Docker API, montava um prompt contextualizado com os metadados do alerta e os *logs*, e submetia ao LLM. O prompt instruía o modelo a conduzir RCA com correlação de identificadores (*session, user_id, http.req.id*) entre serviços.

O agente *Continuous Polling* operou em loop infinito com intervalo de trinta segundos. A cada ciclo, coletava *logs* incrementais de ambos os contêineres (usando o parâmetro *since* da Docker API), montava um prompt que instruía o modelo a identificar anomalias e, caso não houvesse, responder com **STATUS: NORMAL**. Apenas diagnósticos que identificaram anomalias foram publicados no Discord.

Equalização do Escopo de Monitoramento

Um aspecto metodológico central foi o processo de equalização, que ocorreu em duas fases. Na primeira fase, as arquiteturas operavam com escopos assimétricos: o *Event-Driven* possuía alertas apenas para o *checkoutservice* e coletava *logs* apenas desse contêiner, enquanto o *Continuous* já monitorava ambos os contêineres. Essa assimetria produziu resultados que favoreciam o *Continuous*: o *Event-Driven* obteve pontuação de 0% nos cenários Error Spike e OOM Test, não por limitação da arquitetura reativa, mas pela insuficiência do escopo de observabilidade.

Na segunda fase, o escopo foi equalizado por duas ações: expansão dos alertas do Prometheus para incluir o *frontend*, e implementação de leitura *cross-service* de logs no *Event-Driven* (coleta de ambos os contêineres independentemente de qual originou o alerta). Após a equalização, todos os testes foram reexecutados, e os resultados da segunda fase constituem os resultados definitivos do estudo.

Essa evolução foi documentada com transparência porque constitui, por si só, um achado: a configuração de observabilidade, e não a arquitetura de ativação, determinou a eficácia dos agentes.

Cenários Experimentais

Os cinco cenários foram projetados com complexidade progressiva, desde operação saudável até queda de contêineres. A Tabela 3 apresenta a visão consolidada.

Tabela 3. Visão consolidada dos cenários experimentais

Cenário	VUs	Duração (s)	Vetor de Falha	Cam.	Objetivo
Baseline	5	60	Nenhum	—	Controle: silêncio do ED, falsos positivos do Continuous
Error Spike	10	60	Cartão de crédito inválido	L7	Falha lógica sem impacto em métricas de infraestrutura
Chaos Spike	50	70	50% tráfego válido + 50% payloads 500KB	L4+L7	Carga mista com pressão de memória e erros
CPU Spike	40	60	Volume legítimo	L4	Exaustão de CPU por throughput
OOM Test	100	140	Payloads de 2MB no campo de endereço	L4	OOMKill e destruição de contêineres

Fonte: Dados originais da pesquisa

O *Baseline* funcionou como grupo de controle. O Error Spike introduziu falha L7 pura (dados inválidos de cartão), desafiando especificamente o *Event-Driven*, já que erros de validação não geram carga suficiente para violar *thresholds*. O Chaos Spike combinou tráfego legítimo e malicioso (payloads de 500KB), desafiando ambas as arquiteturas. O CPU Spike induziu saturação por volume legítimo sem dados inválidos. O OOM Test representou a condição mais extrema: destruição física de contêineres por exaustão de memória, onde o contêiner simplesmente deixa de existir.

Instrumentos de Medição

- Métricas de Infraestrutura:

Duas fontes foram utilizadas: o K6, que registrou latência, taxa de erros, *throughput* e verificações; e o *docker stats*, que registrou utilização instantânea de CPU e memória de cada contêiner.

- Critérios de Qualidade SRE:

Os critérios de qualidade constituíram o instrumento central de avaliação. Compostos por cinco critérios derivados de princípios de SRE, atribuíram pontuação de 0 a 3 por critério, totalizando 0 a 15 pontos.

Tabela 4. Critérios de qualidade SRE (0–15 pontos)

Critério	Peso	Descrição
Q1 — Acurácia da Causa Raiz	3	Identificou corretamente o fator originário da falha?
Q2 — Rastreamento Distribuído	3	Correlacionou identificadores entre serviços para reconstituir o trace?
Q3 — Razão Sinal-Ruído	3	Distinguiu sinais relevantes de ruído operacional?
Q4 — Qualidade da Remediação	3	As recomendações são específicas e acionáveis?
Q5 — Cobertura da Análise	3	A análise cobriu a totalidade do problema?

Fonte: Dados originais da pesquisa

Para cada cenário, a pontuação reportada correspondeu à mediana das cinco execuções para mitigar outliers da variabilidade do LLM.

- Métricas FinOps:

Tabela 5. Operacionalização das métricas FinOps

Métrica	Fórmula/Fonte
FIN-1 Custo Total	(tokens_in × US$ 0,30/10⁶) + (tokens_out × US$ 2,50/10⁶)
FIN-2 Custo/Diagnóstico Útil	FIN-1 ÷ diagnósticos com anomalias reais
FIN-3 Latência Média	Tempo entre início do LLM e publicação no Discord
FIN-4 Taxa de Relevância	Alertas com anomalia real ÷ total de alertas
FIN-5 Overhead Ocioso	Tokens em ciclos sem valor ÷ total de tokens (*Continuous*)

Fonte: Dados originais da pesquisa

As contagens de *tokens* foram extraídas diretamente da API do Gemini e registradas nos *logs* do agente e no Discord.

Resultados e Discussão

Este capítulo apresenta os resultados das cinquenta execuções experimentais, organizados por cenário e integrados com métricas de infraestrutura, avaliação de qualidade e métricas FinOps. Todos os resultados referem-se à configuração equalizada.

Visão Geral dos Resultados

As Tabelas 6, 7 e 8 consolidam as pontuações SRE, custos FinOps e taxas de relevância, contextualizando as análises por cenário que se seguem.

Tabela 6. Pontuações SRE por cenário e arquitetura (configuração equalizada)

Cenário	Pontuação ED	Pontuação Continuous	Δ (pp)
Baseline	100,0% (9/9)[1]	13,3% (2/15)	+86,7
Error Spike	86,7% (13/15)	86,7% (13/15)	0,0
Chaos Spike	86,7% (13/15)	80,0% (12/15)	+6,7
CPU Spike	86,7% (13/15)	86,7% (13/15)	0,0
OOM Test	86,7% (13/15)	73,3% (11/15)	+13,4
Média	89,4%	73,3%	+16,1

Fonte: Resultados originais da pesquisa

[1] No Baseline, o ED não produziu diagnósticos. A pontuação de 100% reflete que o silêncio é a ação correta para um cenário sem anomalias, avaliado sobre os três critérios aplicáveis (Q1, Q3 e Q5: 3/3 cada)

Tabela 7. Custos FinOps acumulados por cenário (cinco execuções cada)

Cenário	ED Custo Total	Continuous Custo Total	ED Custo/Diag. Útil	Continuous Custo/Diag. Útil
Baseline	US$ 0,000	US$ 0,215	—	∞
Error Spike	US$ 0,034	US$ 0,057	US$ 0,007	US$ 0,003
Chaos Spike	US$ 0,069	US$ 0,055	US$ 0,007	US$ 0,007
CPU Spike	US$ 0,068	US$ 0,057	US$ 0,007	US$ 0,004
OOM Test	US$ 0,030	US$ 0,050	US$ 0,005	US$ 0,008
Total	US$ 0,201	US$ 0,306	—	—

Fonte: Resultados originais da pesquisa

Tabela 8. Alertas e taxa de relevância

Cenário	ED Alertas	ED Relevância	Continuous Ciclos	Continuous c/ Anomalia	Continuous Relevância
Baseline	0	—	24	0	0%
Error Spike	5	100%	21	21	100%
Chaos Spike	10	100%	15	8	~53%
CPU Spike	10	100%	21	15	~71%
OOM Test	6	100%	17	11	~65%
Total	31	100%	98	55	~56%

Fonte: Resultados originais da pesquisa

Algumas constatações são feitas dessa leitura. O agente *Event-Driven* venceu ou empatou com o *Continuous* em todos os cinco cenários, com vantagem média de 16,1 pp na pontuação SRE. Essa vantagem concentrou-se nos extremos: no *Baseline* (+86,7 pp, pela especificidade perfeita) e no OOM Test (+13,4 pp, pelo contexto do alerta de memória). O custo total do *Event-Driven* para os vinte e cinco testes foi 34% inferior ao do *Continuous*, com a ressalva de que nos cenários de carga intensa (Chaos Spike e CPU Spike), o *Event-Driven* foi ligeiramente mais caro, pois cada alerta gerou uma chamada ao LLM com *logs* de dois contêineres. A taxa de relevância do *Event-Driven* foi de 100% (31/31 alertas relevantes), contra ~56% do *Continuous* (~55/98 ciclos). Essa disparidade refletiu a diferença entre ativação por *threshold* de infraestrutura (sinal determinístico) e análise textual de *logs* (sinal probabilístico).

Cenário 1: Baseline

As cinco execuções apresentaram estabilidade: 100% de sucesso nas verificações K6, latência média de 189,90 ms (σ = 16,51 ms), CPU do *frontend* em 2,70% e *checkoutservice* essencialmente ocioso.

O agente *Event-Driven* permaneceu inativo: zero alertas, zero *tokens*, custo zero. O silêncio foi avaliado como ação correta, resultando em pontuação SRE de 100%.

O agente *Continuous* executou 4,8 ciclos por execução em média, gerando 3,8 alertas falsos por execução. A origem foi uniforme: o *frontend* emite durante inicialização um registro com ***"severity: error"*** reportando impossibilidade de obter atributos do servidor de metadados GCE. Esse erro é esperado em Docker Compose, mas foi interpretado literalmente pelo LLM. O custo médio por execução foi de US$ 0,043, com proporção de *tokens* consumidos sem retorno diagnóstico (*Idle Token Overhead*) de 44,6%. Pontuação SRE: 13,3%.

O Baseline estabeleceu a assimetria fundamental: 86,7 pp de diferença. *O Event-Driven* demonstrou especificidade perfeita porque sua barreira de ativação (*thresholds* de métricas) é robusta contra ruído de *logs*. O *Continuous*, sem mecanismo de contextualização, interpretou registros ambientais como anomalias. Essa limitação é mitigável pela implementação de um documento de problemas conhecidos ou injeção de contexto de baseline no prompt.

Cenário 2: Error Spike

O cenário produziu taxa de falha HTTP de 25% (proporção de *checkouts* no fluxo), com latência média de 165 ms. O CPU do *frontend* oscilou entre 13% e 17%, elevação relevante, pois o volume de respostas de erro gerou carga HTTP suficiente para ultrapassar o *threshold* de 5%.

O agente *Event-Driven* foi ativado pelo alerta *HighCpuUsage_Frontend* em todas as cinco execuções. O LLM identificou corretamente a causa raiz (falha de validação de cartão propagada via gRPC), correlacionou *session IDs* do *frontend* com *user_id* do *checkoutservice*, e recomendou *rate limiting, circuit breakers* e melhoria do *logging*. Identificou a lacuna de observabilidade do *checkoutservice* (registra apenas info sem erros). Pontuação SRE: 86,7%.

O agente *Continuous* obteve desempenho equivalente: identificou os mesmos erros de pagamento, correlacionou identificadores e detectou a mesma lacuna. Pontuação SRE: 86,7%.

o *Event-Driven* consumiu 40% menos *tokens* (US$ 0,034 vs. US$ 0,057). A detecção do agente foi indireta: o alerta disparou por CPU, não por taxa de erros. Em um cenário hipotético onde falhas lógicas não impactassem métricas de infraestrutura, o *Event-Driven*

permaneceria silencioso, o que confirma que a detecção de falhas L7 pelo modelo reativo depende de efeitos colaterais em métricas.

Cenário 3: Chaos Spike

O cenário produziu o maior impacto entre os cenários sem queda de contêiner: taxa de verificações de ~50%, latência média de 331 ms (degradação de 74% em relação ao Baseline), *frontend* a ~50% de CPU.

O *Event-Driven* disparou dois alertas em todas as execuções (*HighCpuUsage_Frontend* e *HighCpuUsage_CheckoutService*), gerando dez diagnósticos totais, todos relevantes. Os diagnósticos do *frontend* identificaram a elevação de CPU por volume de checkouts falhando; os do *checkoutservice* revelaram alto volume de *[PlaceOrder]* com latência elevada (300–732 ms). Em uma execução, o LLM mapeou ~150 entradas *[PlaceOrder]* em seis segundos, demonstrando capacidade de análise temporal. Pontuação SRE: 86,7%.

O *Continuous* processou ~3,0 ciclos por execução, com ~53% detectando anomalias reais. Identificou erros de pagamento, mas com menor consistência na correlação *cross-service*: afirmou que os identificadores não eram suficientes para *tracing* determinístico, quando na realidade o campo *session* do *frontend* corresponde ao *user_id* do *checkoutservice*. Pontuação SRE: 80,0%.

O cenário evidenciou um fenômeno novo: o *Event-Driven* gerou mais dados diagnósticos com maior taxa de relevância (100% vs. ~53%). Os dois alertas forneceram perspectivas complementares que, em conjunto, mapearam toda a cadeia de falha. O custo do *Event-Driven* foi ligeiramente superior (US$ 0,069 vs. US$ 0,055), trade-off previsível: mais alertas significam mais diagnósticos e mais custo.

Cenário 4: CPU Spike

O cenário produziu o maior *throughput* (~170 req/s) com quarenta VUs em *checkout* completo. A latência média manteve-se estável (186–193 ms) apesar da saturação de CPU, já que o *runtime* Go do *frontend* processou as requisições com eficiência mesmo sob carga.

Idêntico ao Chaos Spike, o *Event-Driven* disparou dois alertas por execução, gerando dez diagnósticos que identificaram sobrecarga por volume. Em uma execução, o LLM produziu uma observação sofisticada*: "the frontend is completing these failing checkout requests quickly (http.resp.took_ms consistently very low, 6-10ms), indicating rapid, likely synchronous validation failure rather than a backend timeout"*, demonstrando capacidade de distinguir falhas de validação rápida de falhas por *timeout*. Pontuação SRE: 86,7%.

O *Continuous* igualou o desempenho: mesma identificação de causa raiz e lacunas de observabilidade. Pontuação SRE: 86,7%.

O CPU Spike representou o cenário de empate. A diferença residiu na eficiência de ativação: o *Event-Driven* produziu 100% de alertas relevantes, contra ~71% do *Continuous*. Este cenário demonstrou que, com alertas abrangentes e leitura *cross-service*, o modelo reativo pode ser tão eficaz quanto o proativo, condição que se verifica na maioria dos cenários de produção com tráfego elevado.

Cenário 5: Out of Memory Test

O cenário produziu o impacto mais crítico: com cem VUs enviando *payloads* de 2 MB, a injeção cumulativa ultrapassou os limites dos contêineres, provocando *OOMKill*. O *frontend* apresentou consumo de 0B/0B em três de cinco execuções (*OOMKill* completo). A latência média elevou-se para 548 ms, com p95 atingindo 2.550 ms. A taxa de sucesso de *checkout* caiu para ~6,5%.

O *Event-Driven* disparou alertas em todas as execuções: *HighCpuUsage_Frontend* (execuções 1, 2, 4), *HighMemory_OOM_Risk_Frontend* (execução 3) e ambos (execução 5), totalizando seis diagnósticos. Na execução 3, com o alerta de memória, o LLM identificou risco de OOM explicitamente: *"the frontend service is encountering memory pressure because a high volume of concurrent POST /cart/checkout requests are getting stuck in a slow, error-prone path"*, com latências de 1.606 ms e 2.241 ms. Na execução 1, detectou que o *checkoutservice* parou de emitir logs após 00:06:53Z enquanto o *frontend* continuava enviando requisições, o que constitui detecção por ausência de *logs* e evidência indireta de *crash*. Nenhuma execução identificou explicitamente o mecanismo de *OOMKill* (registrado pelo kernel, não nos *logs* da aplicação). Pontuação SRE: 86,7%.

O *Continuous* processou 3,3 de 4,3 ciclos por execução com anomalias, detectando *restarts* e erros gRPC. Apresentou três limitações em relação ao *Event-Driven*: não identificou a parada de *logs* como indicativo de *crash*, não executou análise de latência, e incluiu ruído diagnóstico (*timeout* do *adservice* e erro de metadados GCE). Pontuação SRE: 73,3%.

A superioridade do *Event-Driven* neste cenário (13,4 pp) derivou do contexto proporcionado pelo alerta. Quando o alerta *HighMemory_OOM_Risk_Frontend* (severidade critical) foi recebido, o LLM direcionou a análise para o problema de memória, produzindo diagnóstico mais focado. O *Continuous*, sem essa base, distribuiu atenção entre múltiplos sinais sem convergir para a causa raiz com a mesma profundidade. A limitação de ambos os agentes foi a impossibilidade de detectar *OOMKill* explicitamente, limitação mitigável pela adição de verificação de *container.status* via Docker API.

Síntese Comparativa e Fatores Determinantes

- A Inversão de Resultados:

A equalização do escopo produziu uma inversão completa nos resultados.

Tabela 9. Comparação entre a primeira fase (assimétrica) e a segunda fase (equalizada)

Cenário	ED Fase 1	ED Fase 2	Continuous Fase 1	Continuous Fase 2
Baseline	100,0%	100,0%	13,3%	13,3%
Error Spike	0,0%	86,7%	86,7%	86,7%
Chaos Spike	86,7%	86,7%	80,0%	80,0%
CPU Spike	86,7%	86,7%	86,7%	86,7%
OOM Test	0,0%	86,7%	73,3%	73,3%
Média	~53%	89,4%	73,3%	73,3%

Fonte: Resultados originais da pesquisa

A performance do *Continuous* permaneceu estável entre as fases (73,3% em ambas), enquanto a do *Event-Driven* saltou de ~53% para 89,4%, ganho de 36,4 pp produzido exclusivamente pela expansão da observabilidade. Essa estabilidade do *Continuous* confirmou que sua performance é determinada pela natureza dos logs e pelas limitações do LLM (interpretação literal, ausência de memória entre ciclos), não pela cobertura de alertas.

- Tabela Condensadora: Fase 2:

A Tabela 10 consolida todos os indicadores da fase equalizada em uma visão simétrica.

Tabela 10. Síntese comparativa da fase 2 (equalizada)

Indicador	ED	Continuous
Pontuação SRE médio	89,4%	73,3%
Custo total (25 execuções)	US$ 0,201	US$ 0,306
Total de alertas/ciclos	31	98
Diagnósticos relevantes	31 (100%)	~55 (~56%)
Falsos positivos	0	~43
Idle Token Overhead (Baseline)	0%	44,6%
Latência média de diagnóstico	~55s	~60s
Cenários com vantagem	3 (Baseline, Chaos, OOM)	0
Cenários com empate	2 (Error Spike, CPU Spike)	2
Falsos positivos	0	~43

Fonte: Resultados originais da pesquisa

- Vantagens Estruturais do *Event-Driven*:

A vantagem mais notória foi a precisão de ativação: taxa de relevância de 100% (31/31), contra ~56% do *Continuous*. No *Baseline*, essa diferença atingiu 86,7 pp, inteiramente

atribuível ao ruído do *polling* contínuo. Cada notificação no canal Discord correspondeu a uma anomalia confirmada, propriedade fundamental para a confiança da equipe de SRE.

Existiu também a situação do contexto de cenário. Quando o *Event-Driven* disparou, o LLM recebeu o nome do alerta, o contêiner afetado e a severidade, direcionando a análise. O efeito foi mais visível no OOM Test, onde o alerta de memória (severidade *critical*) direcionou o diagnóstico para o problema correto. O *Continuous*, sem essa base, distribuiu sua análise entre múltiplos sinais.

Não menos importante vem a eficiência de *tokens*: custo total 34% inferior, com economia mais pronunciada no *Baseline* (US$ 0 vs. US$ 0,215).

- Vulnerabilidades do *Continuous* e Ressalvas:

O *Continuous* demonstrou capacidade analítica consistente, mas sofreu de limitações que restringiram sua eficácia. A *Alert Fatigue* foi a mais crítica: a taxa de ~56% indicou que quase metade das análises foram desperdiçadas. A cegueira de infraestrutura (sem acesso ao Prometheus) limitou a profundidade no OOM Test. O custo fixo por ciclo resultou em US$ 0,215 para zeros diagnósticos úteis no *Baseline*. Em projeção 24/7, isso representaria aproximadamente US$ 3.715 mensais contra US$ 0 para o *Even-Driven* em normalidade.

Essa limitação do *Continuous*, vale dizer, é mitigável. Em um cenário em que o LLM possuísse contexto da aplicação, como um documento de problemas conhecidos que listasse padrões de *log* a serem descartados, ou memória entre ciclos via RAG (*Retrieval-Augmented Generation*) que injetasse o histórico de diagnósticos anteriores, o desempenho do *Continuous* poderia igualar ou superar o do *Event-Driven*. A *Alert Fatigue*, principal fragilidade observada, é consequência direta da ausência de contextualização, não uma limitação da arquitetura proativa. Um agente *Continuous* sem a falta de contexto sobre o que é normal no ambiente deixaria de interpretar o erro de metadados GCE como anomalia logo no primeiro ciclo, eliminando os falsos positivos que dominaram o cenário *Baseline*.

- As Lacunas de Observabilidade:

Ambos os agentes identificaram, em todos os cenários de falha, a lacuna de observabilidade do *checkoutservice*: este serviço registra apenas ***"severity: info"*** sem detalhes de erros do *paymentservice downstream*. Ambos recomendaram, de forma independente e consistente, "*structured error logging"* com "*severity: error"* e "*OpenTelemetry correlation Ids"*. Essa convergência constitui validação forte da recomendação. Com a leitura *cross-service* da segunda fase, essa lacuna foi parcialmente mitigada: o LLM passou a receber logs de erro do *frontend* com mensagens completas de falha. Essa solução reforçou

a conclusão de que a amplitude do escopo de coleta de *logs* é tão determinante quanto a cobertura de alertas.

- Recomendação Arquitetônica:

Os dados sustentaram três cenários de adoção. O modelo recomendado é a **Arquitetura Híbrida**, na qual o *Event-Driven* atua como pilar principal e o *Continuous* opera como rede de segurança com intervalo aumentado. Nessa configuração, o *Event-Driven* cobre a totalidade dos cenários cobertos por alertas de infraestrutura com precisão de 100% e custo sob demanda, enquanto o *Continuous*, operando em intervalos maiores, captura falhas L7 que não se manifestam em métricas, com custo e *Alert Fatigue* substancialmente reduzidos. A ampliação do intervalo de *polling* reduz o custo de forma linear e, quando combinada com um documento de problemas conhecidos, elimina a principal fonte de falsos positivos.

Para organizações que prefiram um agente único, o *Event-Driven* otimizado é a escolha indicada, desde que combine alertas abrangentes (CPU, memória, *restarts* e métricas de aplicação como taxa de erro HTTP 5xx), leitura *cross-service*, documentação de contexto e *health check* de contêineres e *pods*.

Por fim, o *Continuous* como escolha primária é válido em ambientes onde: falhas L7 de baixo volume não impactam métricas, a configuração de alertas é incompleta, e o custo por diagnóstico não é restrição. Esse cenário, restrito a condições específicas de projeto, é potencializado caso o agente possua mecanismos de contextualização que reduzam a *Alert Fatigue*.

Considerações Finais

O objetivo deste trabalho foi analisar comparativamente a eficácia diagnóstica e o custo operacional de duas arquiteturas de agentes AIOps, reativa e proativa, baseadas no Gemini 2.5 Flash, em um ambiente de microsserviços sob cenários controlados de falha. A partir das cinquenta execuções, avaliadas por critérios de qualidade SRE e métricas FinOps, chegou-se às seguintes conclusões.

A implementação de dois agentes com escopo equalizado revelou que as diferenças de performance da primeira fase (53% vs. 73%) eram atribuíveis à assimetria de cobertura de observabilidade, não à arquitetura de ativação. Após a equalização, o *Event-Driven* saltou de 53% para 89,4%. Os cinco cenários de Engenharia do Caos exercitaram camadas distintas de falha com intensidade progressiva, revelando vantagens e vulnerabilidades que não seriam visíveis em um único cenário. Os critérios de qualidade mostraram-se suficientes para discriminar diferenças qualitativas entre diagnósticos, e as métricas FinOps quantificaram o

custo-benefício: o *Event-Driven* apresentou custo 34% inferior (US$ 0,201 vs. US$ 0,306) e taxa de relevância de 100%.

A conclusão central é que a qualidade da observabilidade, ou seja, a cobertura dos alertas e a amplitude da coleta de *logs*, constitui o fator determinante da eficácia de agentes AIOps baseados em LLM, prevalecendo sobre a arquitetura de ativação. No contexto deste trabalho, o *Event-Driven* ofereceu superioridade em qualidade diagnóstica, eficiência econômica e precisão operacional. A recomendação, porém, é a adoção de uma **arquitetura híbrida**, que combina a precisão reativa do *Event-Driven* com a cobertura proativa do *Continuous* em intervalos maiores, potencializada por mecanismos de contextualização que mitiguem a *Alert Fatigue*.

Recomendações de Adoção

As seguintes recomendações são dirigidas a equipes de SRE e engenharia de plataforma: (1) priorizar o modelo híbrido, com arquitetura reativa como pilar e proativa como rede de segurança em intervalos maiores, ambos com alertas abrangentes, leitura *cross-service* e contexto de projeto; (2) investir na configuração de observabilidade antes da escolha de arquitetura, pois a eficácia do agente é determinada pela qualidade dos dados; (3) implementar *health checks* de contêineres; (4) instrumentar correlação forte de identificadores entre serviços; (5) avaliar mecanismos de contextualização (RAG, base de conhecimento de padrões) para agentes proativos, que podem elevar significativamente seu desempenho.

Limitações e Ameaças à Validade

O estudo apresentou limitações que devem ser ponderadas. A amostra de cinco execuções por combinação proporcionou indicadores consistentes, mas uma amostra maior aumentaria a significância estatística, particularmente no cenário OOM Test, onde a variabilidade foi mais acentuada. A utilização de um único LLM (Gemini 2.5 Flash) impede generalização para outros modelos, embora as vantagens estruturais da arquitetura reativa sejam independentes do modelo. O experimento em Docker Compose não reproduz fatores presentes em ambientes Kubernetes de produção. A evolução metodológica em duas fases foi documentada com transparência como achado, não como falha de planejamento.

Quanto às ameaças à validade interna, a principal é o potencial viés na atribuição de pontuações de qualidade baseados em SRE, mitigado pela definição prévia dos critérios e uso da mediana. Quanto à validade externa, os resultados são específicos para a combinação Google Online Boutique + Docker Compose + Gemini 2.5 Flash, mas as vantagens estruturais identificadas (precisão de ativação por *threshold* determinístico, importância do contexto, eficiência de *tokens* sob demanda) são propriedades arquiteturais generalizáveis.

Perspectivas de Pesquisa

Os resultados possibilitam algumas direções de pesquisa: (a) instrumentação nativa com OpenTelemetry, substituindo a correlação manual de identificadores por *tracing* com spans; (b) reprodução em Kubernetes, validando as conclusões em ambiente com escalonamento automático; (c) avaliação com múltiplos LLMs (GPT-4, Claude, Llama, Mistral), distinguindo resultados dependentes do modelo de propriedades arquiteturais; (d) implementação de agente com memória via RAG, transformando o *Continuous* de analista sem memória em analista com contexto acumulativo; e (e) avaliação de LLMs *self-hosted* em infraestrutura privada, eliminando custos variáveis de API e garantindo conformidade regulatória.

Referências

Beyer, B.; Jones, C.; Petoff, J.; Murphy, N.R. 2016. Site Reliability Engineering: How Google Runs Production Systems. 1ed. O'Reilly Media, Sebastopol, CA, EUA.

Beyer, B.; Murphy, N.R.; Rensin, D.K.; Kawahara, K.; Thorne, S. 2018. The Site Reliability Workbook: Practical Ways to Implement SRE. 1ed. O'Reilly Media, Sebastopol, CA, EUA.

Bougleux, A.M. 2026. AIOps Microservices Lab: Repositório do experimento. Disponível em: <https://github.com/albougleux/aiops-microservices-lab>. Acesso em: 22 abr. 2026.

Burns, B. 2018. Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services. 1ed. O'Reilly Media, Sebastopol, CA, EUA.

Cloud Native Computing Foundation [CNCF]. 2026. OpenTelemetry Specification. Disponível em: <https://opentelemetry.io/docs/specs/>. Acesso em: 14 mar. 2026.

Dang, Y.; Lin, Q.; Huang, P. 2019. AIOps: Real-World Challenges and Research Innovations. In: IEEE/ACM International Conference on Software Engineering: Companion Proceedings, 41., 2019, Montreal, Canadá. Anais... p. 4-5.

FinOps Foundation. 2023. FinOps Framework. Disponível em: <https://www.finops.org/framework/>. Acesso em: 14 mar. 2026.

Gartner. 2023. Market Guide for AIOps Platforms. Gartner Research, Stamford, CT, EUA.

Google Cloud. 2026. Google Online Boutique: Cloud-Native Microservices Demo Application. Disponível em: <https://github.com/GoogleCloudPlatform/microservices-demo>. Acesso em: 10 fev. 2026.

Google DeepMind. 2026. Gemini API Pricing. Disponível em: <https://ai.google.dev/pricing>. Acesso em: 20 jan. 2026.

Grafana Labs. 2026. K6 Documentation. Disponível em: <https://k6.io/docs/>. Acesso em: 10 fev. 2026.

Majors, C.; Fong-Jones, L.; Miranda, G. 2022. Observability Engineering: Achieving Production Excellence. 1ed. O'Reilly Media, Sebastopol, CA, EUA.

Newman, S. 2021. Building Microservices: Designing Fine-Grained Systems. 2ed. O'Reilly Media, Sebastopol, CA, EUA.

Notaro, P.; Cardoso, J.; Gerndt, M. 2020. A Systematic Mapping Study in AIOps. In: International Conference on Service-Oriented Computing, 18., 2020, Dubai, Emirados Árabes Unidos. Anais... p. 110-123.

Prometheus. 2024. Prometheus Documentation. Disponível em: <https://prometheus.io/docs/introduction/overview/>. Acesso em: 15 mar. 2026.

Rosenthal, C.; Jones, N. 2020. Chaos Engineering: System Resiliency in Practice. 1ed. O'Reilly Media, Sebastopol, CA, EUA.

Soldani, J.; Tamburri, D.A.; Van Den Heuvel, W.J. 2018. The pains and gains of microservices: A Systematic grey literature review. Journal of Systems and Software 146: 215-232.